

Sistemas embebidos

Entorno de desarrollo y programación básica

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2026-II

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

- 1 Instalación y configuración de Arduino IDE
- 2 Estructura de un programa para ESP32
- 3 Variables y tipos de datos
- 4 Operadores aritméticos, relacionales y lógicos
- 5 Comentarios y buenas prácticas de programación



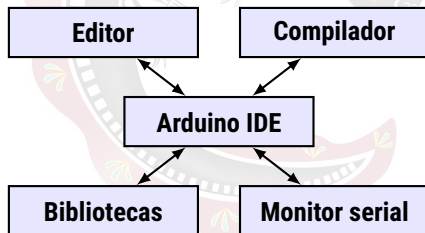
Arduino IDE

Definición

Un **IDE (Integrated Development Environment)** es una **aplicación** que reúne las **herramientas necesarias** para **escribir, compilar, cargar y depurar** programas.

Arduino IDE proporciona:

- **Editor de código.**
- **Compilador y ensamblador.**
- **Carga** del **firmware**.
- Administrador de **tarjetas**.
- Administrador de **bibliotecas**.
- **Monitor y graficador** serial.



Instalación de Arduino IDE

Windows

- 1 **DESCARGA LA ÚLTIMA VERSIÓN**  del IDE.
- 2 **Ejecuta y sigue las instrucciones** de instalación.

GNU/Linux

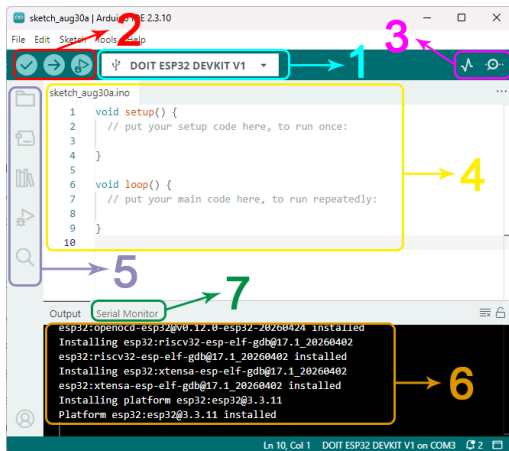
- 1 **DESCARGA LA ÚLTIMA VERSIÓN**  del IDE. 
- 2 **Localiza** el archivo [AppImage](#) con el explorador de archivos.
- 3 **Cambia los permisos** del archivo de tal forma que sea [ejecutable](#).
- 4 **Ejecuta el archivo**

Nota: en algunos casos es necesario instalar `libfuse`

¡Importante!

En **GNU/Linux** es necesario [verificar](#) que se tengan los **permisos necesarios** para acceder al puerto serie, para ello **DESCARGA ESTE SCRIPT**  y ejecútalo.

Interfaz de Arduino IDE

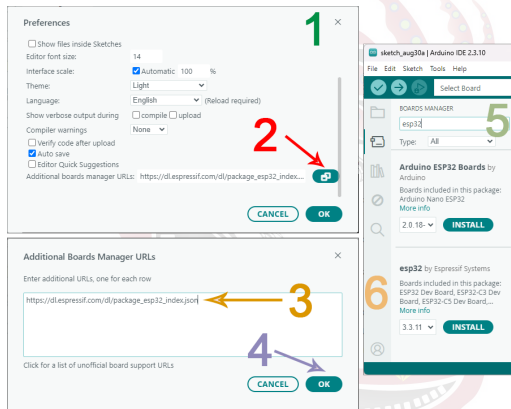


Elementos principales

- 1 Selector de **tarjeta y puerto**.
- 2 Botones de **verificación, carga y depuración**.
- 3 Botones de graficador y monitor serial.
- 4 **Editor** del programa.
- 5 **Proyecto**, selector de tarjeta, **bibliotecas**, depuración y búsqueda.
- 6 Consola de **compilación**.
- 7 Consola de **monitor serial**.

Instalar el soporte para ESP32

- 1 *File → Preferences.*
- 2 **Seleccionar** *Additional board manager URLSs*
- 3 Colocar la dirección.
- 4 *OK.* en **ambos diálogos.**
- 5 **Buscar** esp32 en *Boards Manager* y
- 6 **Instalar** esp32 by Expressif Systems.



Dirección

https://dl.espressif.com/dl/package_esp32_index.json

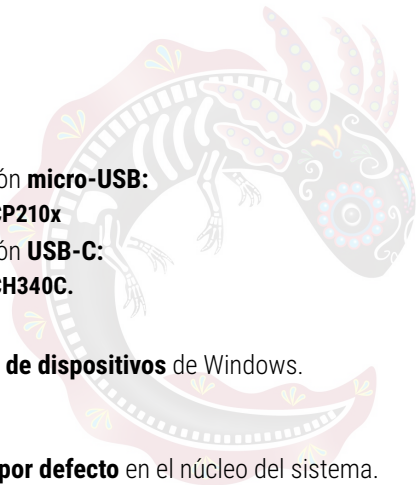
Instalación del controlador

Windows

- 1 **Identifica** el tipo de placa.
- 2 Si tu placa cuenta con puerto de conexión **micro-USB**:
 - **DESCARGA**  e instala el controlador **CP210x**
- 3 Si tu placa cuenta con puerto de conexión **USB-C**:
 - **DESCARGA**  e instala el controlador **CH340C**.
- 4 **Conecta la tarjeta** al puerto USB.
- 5 **Verifica la conexión** en el **administrador de dispositivos** de Windows.

GNU/Linux

Los controladores CP210x y CH340C **vienen por defecto** en el núcleo del sistema.



Configuración de la ESP32 DevKit V1

Antes de cargar un programa es necesario configurar correctamente la tarjeta.

1. Conexión

Conectar la ESP32 mediante un cable USB que permita **transmisión de datos**.

2. Selección de la tarjeta

En arduino IDE, seleccionar la tarjeta **DOIT ESP32 DEVKIT V1**

3. Puerto

Seleccionar el **puerto serial** asociado con la interfaz USB de la tarjeta.

4. Carga

Compilar y transferir el programa a la memoria flash de la tarjeta.

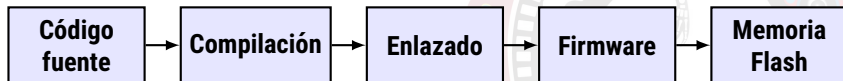
¡Importante!

Algunos cables USB únicamente proporcionan alimentación y **no permiten transferencia de datos**.

Del código fuente a la ESP32

Definición

El programa escrito por el desarrollador debe transformarse en instrucciones ejecutables por el procesador de la ESP32 antes de almacenarse en la tarjeta.



- **Compilación:** transforma el código fuente.
- **Enlazado:** integra código y bibliotecas.
- **Firmware:** programa ejecutable para el sistema embebido.
- **Carga:** transfiere el firmware a la memoria Flash.

Primer programa: Blink

Código

```
1  const int LED = 2;
2
3  void setup() {
4      pinMode(LED, OUTPUT);
5  }
6
7  void loop() {
8      digitalWrite(LED, HIGH);
9      delay(1000);
10
11     digitalWrite(LED, LOW);
12     delay(1000);
13 }
```

Funcionamiento

- 1 **Configura** el **GPIO** como **salida**.
- 2 **Enciende** el **LED**.
- 3 **Espera** 1 segundo.
- 4 **Apaga** el **LED**.
- 5 **Espera** 1 segundo.
- 6 **Repite** el proceso.

¡Importante!

El **número de pin** del **GPIO** debe **adaptarse** al pin donde se encuentre **conectado el LED físicamente**.

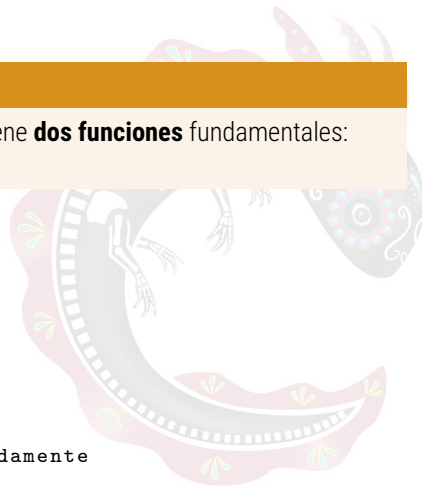
Estructura de un programa

Definición

Un **programa** desarrollado en **Arduino** contiene **dos funciones** fundamentales: `setup()` y `loop()`.

Estructura básica

```
1 // Declaraciones globales
2
3 void setup() {
4     // Inicializacion
5 }
6
7 void loop() {
8     //Codigo que se ejecuta repetidamente
9 }
```



Función `setup()`

Definición

La **función** `setup()` se ejecuta **una sola vez** después de **encender o reiniciar** la ESP32. Se utiliza principalmente para realizar la **configuración inicial** del sistema.

Usos frecuentes

- Configurar **GPIO**.
- Inicializar **UART**.
- Inicializar **sensores**.
- Configurar **periféricos**.
- Establecer **comunicaciones**.

Ejemplo

```
1 void setup() {  
2     Serial.begin(115200);  
3     pinMode(2, OUTPUT);  
4 }
```

¡Importante!

`setup()` establece el **estado inicial** necesario antes de **comenzar** la **operación normal** del sistema.

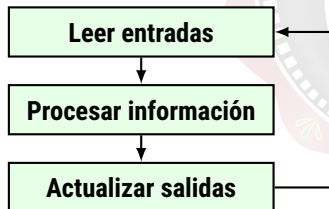
Función `loop()`

Definición

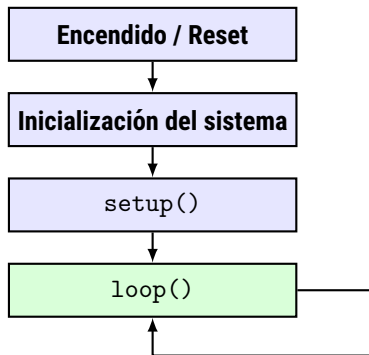
Después de **finalizar** `setup()`, la **función** `loop()` se ejecuta **repetidamente** mientras el sistema **permanezca funcionando**.

Modelo básico

Una gran cantidad de sistemas embebidos pueden describirse mediante el ciclo:



Flujo de ejecución



Secuencia

- 1 La ESP32 se **enciende** o **reinicia**.
- 2 Se **inicializa el hardware** y el entorno de ejecución.
- 3 Se ejecuta `setup()` **una vez**.
- 4 Se ejecuta `loop()` **repetidamente**.

Monitor serial

Definición

El **monitor serial** permite **intercambiar información** entre la ESP32 y la computadora mediante una **comunicación serial**.

Ejemplo

```
1 void setup() {  
2     Serial.begin(115200);  
3 }  
4  
5 void loop() {  
6     Serial.println("ESP32 funcionando");  
7     delay(1000);  
8 }
```

Aplicaciones

- Mostrar **variables**.
- Visualizar **sensores**.
- Comprobar **estados**.
- Mostrar **mensajes**.
- Identificar errores.

¡Importante!

La **velocidad configurada** en el **monitor serial** debe **coincidir** con la especificada mediante **Serial.begin()**, por ejemplo, **115200 baudios**.

Variables

Definición

Una **variable** es una **región de memoria** identificada mediante un **nombre**, utilizada para **almacenar** un valor que **puede cambiar** durante la ejecución del programa.

Para **utilizar** una variable se debe **especificar** su **tipo de dato** y un **identificador**.

tipo **nombre** = **valor**;

Declaración

```
1 int contador;  
2 float voltaje;  
3 bool alarma;
```

Inicialización

```
1 int contador = 0;  
2 float voltaje = 3.3;  
3 bool alarma = false;
```

¡Importante!

Siempre que sea posible, **inicializar las variables** antes de utilizarlas.

Tipos de datos básicos

Definición

El **tipo de dato** determina qué **información** puede **almacenar** una variable y **cómo** será **interpretada** por el programa.

Tipo	Ejemplo	Uso típico
<code>bool</code>	<code>true</code>	Estados lógicos
<code>char</code>	<code>'A'</code>	Caracteres
<code>int</code>	<code>-25</code>	Números enteros
<code>unsigned int</code>	<code>100</code>	Enteros sin signo
<code>long</code>	<code>100000L</code>	Enteros grandes
<code>float</code>	<code>23.5</code>	Números reales (precisión simple)
<code>double</code>	<code>3.14159</code>	Números reales (precisión doble)

Tipos enteros de tamaño fijo

Tipo	Bits	Rango
<code>int8_t</code>	8	-128 a $+127$
<code>uint8_t</code>	8	0 a 255
<code>int16_t</code>	16	-32768 a $+32767$
<code>uint16_t</code>	16	0 a 65535
<code>int32_t</code>	32	-2^{31} a $+2^{31} - 1$
<code>uint32_t</code>	32	0 a $2^{32} - 1$

¡Importante!

En **sistemas embebidos** resulta **útil** especificar **explícitamente** el **número de bits** utilizado para **representar un dato**.

Constantes

Definición

Una **constante** representa un **valor** que **no debe modificarse** durante la ejecución del programa.

Variable

```
1 int led = 2;  
2 led = 4;
```

Constante

```
1 const int LED = 2;
```

También es posible definir constantes utilizando la directiva del preprocesador **#define**

¡Importante!

Las **constantes** hacen **explícito** qué valores forman parte de la **configuración del sistema** y ayudan a evitar modificaciones accidentales.

Operadores aritméticos

Definición

Los **operadores aritméticos** permiten **realizar cálculos** utilizando **variables** y **constantes**.

Operador	Operación	Ejemplo
+	Suma	$a + b$
-	Resta	$a - b$
*	Multiplicación	$a * b$
/	División	a / b
%	Módulo	$a \% b$

¡Importante!

El **resultado** de una **operación** depende de los **tipos de datos** de sus **operandos**.
Una división entre enteros produce un resultado entero.

Operadores de asignación

Definición

El operador `=` permite **almacenar** el resultado de una **expresión** en una variable.

Asignación

```
1 contador = 10;  
2 contador = contador + 5;  
3 contador = contador - 2;
```

Incremento

```
1 contador++;
```

Equivale, en este contexto, a:

```
1 contador = contador + 1;
```

Asignación compuesta

```
1 contador += 5;  
2 contador -= 2;  
3 contador *= 3;  
4 contador /= 2;
```

Decremento

```
1 contador--;
```

Operadores relacionales

Definición

Los **operadores relacionales** permiten **comparar dos valores**. El resultado de la comparación es un **valor lógico**: **true** o **false**.

Operador	Significado	Ejemplo
==	Igual que	x == 10
!=	Diferente de	x != 10
>	Mayor que	x > 10
<	Menor que	x < 10
>=	Mayor o igual que	x >= 10
<=	Menor o igual que	x <= 10

Operadores lógicos

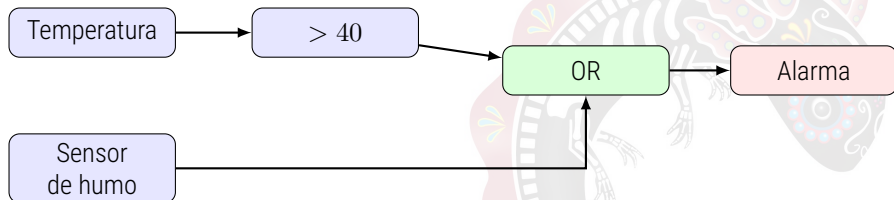
Definición

Los **operadores lógicos** permiten **combinar o negar** expresiones que producen valores **true** o **false**.

Operador	Operación	Resultado
&&	AND	Verdadero si ambas condiciones son verdaderas
	OR	Verdadero si al menos una condición es verdadera
!	NOT	Invierte el valor lógico

Expresiones en un sistema embebido

Los operadores **permiten transformar** los **datos obtenidos** del hardware en **condiciones** que **representan el estado** del sistema.



Ejemplo

```

1 bool alarma;
2 alarma = temperatura > 40.0 || humoDetectado;

```

La alarma debe activarse cuando:

- La temperatura supera el límite, **o**
- El sensor detecta humo.

Comentarios

Definición

Los **comentarios** permiten **documentar** el **código fuente** y son **ignorados** durante la compilación.

Una línea

```
1 // Inicializa el puerto serial
2 Serial.begin(115200);
```

Varias líneas

```
1 /*
2  Configuración inicial
3  de los periféricos
4  */
```

¡Importante!

Un **buen comentario** debe **aportar información** que **no resulte evidente** únicamente al leer el código.

Buenas prácticas de programación

Un **programa** para un **sistema embebido** debe ser **legible, mantenible** y **fácil de verificar**.

Código poco descriptivo

```
1 int x = 2;  
2 int y = 500;  
3  
4 digitalWrite(x, 1);  
5 delay(y);
```

Código más claro

```
1 const int LED_ESTADO = 2;  
2 const int TIEMPO_PARPADEO = 500;  
3  
4 digitalWrite(LED_ESTADO, HIGH);  
5 delay(TIEMPO_PARPADEO);
```

Recomendaciones

- Utilizar **nombres descriptivos**.
- Mantener una **indentación consistente**.
- Utilizar **constantes** para **valores que no cambian**.
- Evitar *números mágicos*.
- Dividir **problemas complejos** en **funciones**.
- **Comentar decisiones relevantes**, no cada instrucción.

Ejemplo integrador

Programa

```

1  const int LED = 2;
2  const unsigned long INTERVALO = 1000;
3
4  void setup() {
5      Serial.begin(115200);
6      pinMode(LED, OUTPUT);
7  }
8
9  void loop() {
10     digitalWrite(LED, HIGH);
11     Serial.println("LED encendido");
12     delay(INTERVALO);
13
14     digitalWrite(LED, LOW);
15     Serial.println("LED apagado");
16     delay(INTERVALO);
17 }

```

Identificar

- 1 Constantes.
- 2 **Tipos de datos.**
- 3 `setup()`.
- 4 `loop()`.
- 5 Llamadas a **funciones.**
- 6 Argumentos.
- 7 **Salida digital.**
- 8 **Comunicación serial.**



Fin de la presentación

¿Preguntas o comentarios?

